

C PROGRAMMING DEBUGGING TOOLS

Project Report

Submitted by:- Reyansh pandit

Sap ID :-590033602

Batch:-16

Tutorial: C Programming - Debugging Tools

1. Fundamentals of Errors and Debugging

Term	Meaning	Example
Error	A mistake in a program that causes it to fail or produce an incorrect result.	Missing ; in C
Bug	A flaw or mistake in the program's logic that makes it behave incorrectly.	Using + instead of * in a calculation
Debugging	The process of finding, understanding, and fixing bugs/errors in a program.	Checking the code to find why the calculation is wrong

2. Classifications of Errors in C

Type of Error	Description	Primary Cause	Example in C
Syntax Error	Occurs when the program does not follow the rules/syntax of the C language.	Missing/incorrect symbols, keywords, brackets, semicolons, etc.	<code>printf("Hello")</code> — missing ;
Compilation Error	Occurs when the compiler cannot convert the source code into object/machine code.	Syntax errors, undeclared variables, incorrect data types, etc.	<code>int x = "Hello";</code> or using an undeclared variable
Linker Error	Occurs after compilation when the linker cannot connect all required	Missing function definition, missing library, or incorrect linking.	Calling <code>calculate()</code> without defining it

Type of Error	Description	Primary Cause	Example in C
	program components.		
Runtime Error	Occurs while the program is actually running.	Invalid operations, memory problems, division by zero, etc.	int x = 10 / 0;
Logical Error	The program runs successfully but produces the wrong result.	Incorrect logic, formula, condition, or algorithm.	Using a + b when the required operation is a * b

Q)Why Does a Program Compile Successfully but Produce Incorrect Output?

:- A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert that code into machine instructions. The compiler cannot understand human intent. • If you write `sum = a - b;` instead of `sum = a + b;` the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

When a compiler detects a problem, its diagnostic commonly contains the source filename, line number, column

number, message category, and explanation. Learning to read these messages is an important programming skill.

```

(kali@kali)~/demo1
$ more error1.c
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}

(kali@kali)~/demo1
$ cc error1.c
error1.c: In function 'main':
error1.c:5:9: error: expected '(', or ';' before 'printf'
5 |     printf("%d\n", a);
  |     ^~~~~~
(kali@kali)~/demo1
$

```

Filename: Error1.c

Line and column: line 5 & column 5

Error/warning: indicates the diagnostic severity.

Description: Expected , or ; before printf statement.

Case 2: Compiler Warnings (-Wall, -Wextra)

Warnings are messages about code that the compiler considers suspicious, questionable, or potentially unsafe even

though it may still be legal to compile. Warnings are extremely useful because they can expose problems before the program reaches runtime.

-Wall enables a broad set of commonly useful warnings. **-Wextra** enables additional warnings that are not all covered by **-Wall**. For student work and professional C development, compiling with warning options is a valuable habit.

```
(kali@kali)-[~/demo1]
$ more error2.c
#include <stdio.h>
int main()

    int a;
    printf("Hello\n");
    return 0;

(kali@kali)-[~/demo1]
$ cc -Wall error2.c
error2.c: In function 'main':
error2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
   4 |         int a;
     |         ^
(kali@kali)-[~/demo1]
$
```

The variable unused may trigger an unused-variable warning when suitable warning flags are enabled. The program

may still run, but the warning tells the programmer that the declaration deserves attention.

Filename: Error2.c

Line and column: line 3 & column 3

Error/warning: indicates the diagnostic severity.

Description: gives a clue about what the compiler expected or detected.

Case-3: printf-Based Debugging (Logical bug)

printf-based debugging is a simple method in which temporary output statements are inserted to observe values and

execution flow. It is especially useful for beginner programmers because it requires no debugger interface.

What can be inspected? Variable values, loop counters, function arguments, intermediate calculations, branches

taken, and whether a particular section of code is reached.

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

```
printf("DEBUG: i = %d, sum = %d\n", i, sum);
```

For a loop, printing the loop counter and accumulated result can reveal an incorrect starting value, stopping condition,

or update expression.

A terminal window on a Kali Linux system. The prompt is (kali@kali)~[~/demo1]. The user runs '\$ more error3.c' to view a C program. The program includes <stdio.h> and has a main function that initializes i=0 and sum=0, then enters a for loop from i=0 to i=4. Inside the loop, it prints 'DEBUG: i = %d\n', increments sum by i, and prints 'DEBUG: sum = %d\n'. After the loop, it prints 'Final Sum = %d\n' and returns 0. The user then runs '\$ cc error3.c' to compile the program. Finally, the user runs '\$./a.out' to execute it. The output shows the debug messages for each iteration of the loop, with the final sum being 10.

```
(kali@kali)~[~/demo1]
$ more error3.c
#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: i = %d\n" , i);
        sum = sum + i;
        printf("DEBUG: sum = %d\n" , sum);
    }
    printf("Final Sum = %d\n" , sum);
    return 0;
}

(kali@kali)~[~/demo1]
$ cc error3.c

(kali@kali)~[~/demo1]
$ ./a.out
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
```

Limitation: Too many print statements can make output difficult to read and can sometimes affect timing or program

behaviour. Once a program becomes complex, a debugger such as GDB is generally more suitable.

- **Introduction to GDB (GNU Debugger)**

- **GNU:** GNU stands for “GNU's Not Unix!” and is associated with a large free-software project begun by Richard
- Stallman in 1983. GNU provides many tools used in Unix-like development environments.
- **GDB:** The GNU Debugger is a tool for examining a program while it runs. It can pause execution, inspect variables,
- move through statements, examine the call stack, and investigate crashes.
- GDB is particularly useful when a program is too complicated to understand using only print statements. Instead of
- modifying the source repeatedly, the programmer can pause at a selected point and inspect the existing program
- state.

Installation & Compilation for GDB #

Update package list and install GDB on Linux systems \$ sudo apt update \$ sudo apt install gdb # Compile source code with debugging symbols enabled (-g) \$ cc -g -Wall -Wextra program.c -o program # Launch GDB with the compiled binary \$ gdb ./program.

Why compile with -g?

:-Without debugging information, GDB can still inspect some low-level information, but source-level debugging becomes much less convenient. With -g, GDB can associate execution with source lines and make variables easier to inspect.

GDB Breakpoints

A breakpoint is a deliberate stopping point in a program. When execution reaches a breakpoint, GDB pauses the program before executing the selected statement. The programmer can then inspect variables and decide what to do next.

```
gdb ./program
(gdb) break main
(gdb) run
```

Breakpoints can be placed at function names or source lines. They are useful when you know approximately where the incorrect behavior occurs and want to inspect the program immediately before it happens.

Example: If a variable becomes incorrect inside a loop, place a breakpoint at the statement that updates it. Inspect

the variable, continue execution, and inspect it again on later iterations.

Important GDB Commands – Detailed Guide

run (r): Starts program execution from the beginning or restarts it.

break (b): Creates a breakpoint.

print (p): Displays the current value of an expression.

display: Automatically displays an expression after stops/steps.

continue (c): Resumes execution until another stopping point.

list (l): Shows source code around the current location.

info locals: displays local variables in the current stack frame.

backtrace (bt): displays the call stack. **info**

breakpoints: shows breakpoints and their status. **delete:** removes a breakpoint. **disable/enable:** temporarily turns a breakpoint off or on. **quit:** exits GDB.

Why commands matter: A debugger is most effective when you use commands with a clear question in mind.

For example, use `print i` to answer “What is the loop counter right now?” rather than stepping randomly through the program.

Case 4: Debugging a Logical Error using GDB

```
Breakpoint 1, main () at error4.c:7
7       sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.
Sum = 3

Breakpoint 1, main () at error4.c:7
7       sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.
Sum = 6

Breakpoint 1, main () at error4.c:7
7       sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
```

- Expected Output: 15 (1+2+3+4+5)
- Actual Output: 10 (1+2+3+4)

Case 5: Debugging Functions (next vs. step)

Both commands move execution forward, but they differ when the current line contains a function call.

next (n): steps over a function call. If the current line calls a function, GDB executes the function and returns to the caller without entering the function body line-by-line.

```
Reading symbols from ./error5...
(gdb) list
4       int result1;
5       result1 = x * x;
6       return result1;
7
8       int main()
9       {
10          int result, n = 5;
11          result = square(n);
12          printf("%d\n", result);
13          return 0;
14      }
(gdb) break main
Breakpoint 1 at 0x1156: file error5.c, line 9.
(gdb) run
Starting program: /home/kali/demo1/error5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error5.c:9
9      {int result, n = 5;
(gdb) next
10          result = square(n);
(gdb) next
11          printf("%d\n", result);
(gdb) print result
$1 = 25
(gdb)
```

step (s): steps into a function call. GDB enters the called function, allowing you to inspect its parameters, local variables, and statements.

```
Breakpoint 1, main () at error5.c:9
9      {int result, n = 5;
(gdb) next
10          result = square(n);
(gdb) step
square (x=5) at error5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result
X No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb)
```

If you trust square() and only want its result, use next. If you suspect the multiplication or the function's internal state, use step.

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.


```
(kali@kali)-[~/demo1]
$ cc -g error6.c -o error6

(kali@kali)-[~/demo1]
$ gdb ./error6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://
gnu.org/licenses/gpl.html>
This is free software: you are free to change and
redistribute it.
There is NO WARRANTY, to the extent permitted by l
aw.
Type "show copying" and "show warranty" for detail
s.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration detail
s.
For bug reporting instructions, please see:
--Type <RET> for more, q to quit, c to continue wi
thout paging--
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resour
ces online at:
<http://www.gnu.org/software/gdb/documentation
/>.

For help, type "help".
Type "apropos word" to search for commands related
to "word"...
Reading symbols from ./error6...
```

```
(gdb) list
6      int calculate(int x)
7      {
8          return divide(x,0);
9      }
10     int main()
11     {
12         int result = calculate(10);
13         printf("Result:%d\n", result);
14         return 0;
15     }
(gdb) run
Starting program: /home/kali/demo1/error6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic excepti
on.
0x0000555555555147 in divide (a=10, b=0)
    at error6.c:4
4      return a/b;
(gdb) backtrace
#0  0x0000555555555147 in divide (a=10, b=0)
    at error6.c:4
#1  0x0000555555555166 in calculate (x=10)
    at error6.c:8
#2  0x000055555555517a in main () at error6.c:12
(gdb) frame 0
#0  0x0000555555555147 in divide (a=10, b=0)
    at error6.c:4
```

```
(gdb) frame 0
#0  0x0000555555555147 in divide (a=10, b=0)
    at error6.c:4
4      return a/b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x0000555555555166 in calculate (x=10)
    at error6.c:8
8      return divide(x,0);
(gdb) print x
$2 = 10
(gdb)
```

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).

- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

Case 7: Debugging Array Out-of-Bounds Errors using GDB

C arrays have fixed storage determined by their declared size, but C does not automatically perform bounds checking

for ordinary array indexing. For `int a[5]`, only indices 0 through 4 belong to the array.

`int a[5];`

The loop executes for `i = 0, 1, 2, 3, 4`, and 5. The access `a[5]` is outside the declared array. This is undefined

behaviour and may overwrite unrelated memory or cause other failures.

```
to "word" ...
Reading symbols from ./error7 ...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4          int a[5];
5          int i;
6          for(i = 0; i ≤ 5; i++)
7          {
8              scanf("%d", &a[i]);
9          }
10         return 0;
(gdb) break 8
Breakpoint 1 at 0x114a: file error7.c, line 8.
(gdb) run
Starting program: /home/kali/demo1/error7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error7.c:8
8          scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd10
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd20
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd24
(gdb) █
```

Correct loop:

```
for (i = 0; i < 5; i++) {
    scanf("%d", &a[i]);
}
```

Debugging approach: inspect the loop condition, print the index, place a breakpoint at the array access, and check

whether the index is within the valid range before the access occurs.

Case 8: Segmentation Faults (SIGSEGV) using GDB

A segmentation fault is a common operating-system-level indication that a process attempted an invalid memory

access. One common cause is dereferencing a NULL pointer.

The pointer `p` contains a null address. Dereferencing it with `*p` attempts to access memory through that invalid

pointer.

```
cc -g sample8.c -o sample8
```

```
gdb ./sample8
```

```
(gdb) run
```

Program received signal SIGSEGV, Segmentation fault.

```
(gdb) print p
```

```
(gdb) print *p
```

GDB may show `p = 0x0`. Attempting to print `*p` can itself fail because the pointed-to memory is not valid.

```
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./error8...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4          int *p = NULL;
5          *p = 10;
6          printf("%d\n", *p);
7          return 0;
8      }
(gdb) run
Starting program: /home/kali/demo1/error8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00005555555514d in main () at error8.c:5
5          *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
X Cannot access memory at address 0x0
```

Corrected approach:

```
int x;  
int *p = &x;  
*p = 10;  
printf("%d\n", *p);
```

Now p points to the valid storage occupied by x, so writing through p is valid.

Systematic Eight-Step Debugging Strategy

1. Observe expected vs actual output — Write down what the program should produce and what it actually

produces. Avoid vague statements such as “it is not working.”

2. Reproduce the issue consistently — Find inputs and conditions that reliably trigger the bug. Consistent

reproduction makes investigation much easier.

3. Compile with -Wall -Wextra -g — Warnings may reveal suspicious code, while -g provides information for source-level GDB debugging.

4. Isolate suspicious code — Reduce the problem to the smallest relevant function, loop, condition, or statement.

5. Inspect state — Use printf for simple tracing or GDB commands such as break, print, display, and info locals.

6. Apply one minimal fix — Change one relevant thing at a time. Large uncontrolled changes make it difficult to know what actually solved the problem.

7. Recompile and re-test — A source change must be rebuilt. Then run the same test that originally exposed the problem.

8. Verify edge cases and boundaries — Test zero, negative values, maximum/minimum values, empty input,

first/last loop iterations, and other boundary conditions as appropriate.

Final Revision Summary

The most important idea is that different errors appear at different stages.

Syntax and many type/declaration

problems are found during compilation. Linker errors occur when compiled components cannot be combined.

Runtime errors appear while the executable is running. Logical errors are different because the program may run

normally while still doing the wrong thing.

Debugging is therefore an evidence-based process. Start with the expected and actual behavior, reproduce the

problem, inspect warnings, isolate the suspicious code, inspect variables and execution flow, make a controlled

correction, rebuild, and test again. For simple problems, printf tracing can be enough; for deeper problems, GDB

provides breakpoints, stepping, variable inspection, stack-frame analysis, and crash investigation.

One-line memory trick: *Compiler finds many code mistakes ® Linker connects program parts ® Runtime*

reveals execution failures ® Logic testing reveals wrong answers ®

Debugging finds and fixes the underlying

problem.